Membagi Satu Berkas Menjadi Lima yang Jelas Tugasnya

Pengembangan Aplikasi Berbasis Web · SIF302 · Semester Gasal 2026/2027

NAMA	MUCHAMAD ADI PRASETYO	NIM	25523259
KELAS	A	TANGGAL	19/10/2026

Dikerjakan sendiri, bukan berpasangan. Setiap mahasiswa melanjutkan halaman profilnya sendiri. Anda bebas memilih isi halaman; yang dinilai adalah apakah tanggung jawab setiap berkas jelas dan apakah halaman mengingat pilihan pengguna.

Capaian pembelajaran	Yang dinilai hari ini
Mahasiswa mampu memecah berkas JavaScript menurut tanggung jawab, memisahkan keadaan aplikasi dari tampilan, dan menyimpannya di peramban sehingga halaman mengingat pilihan pengguna.	1. Lima modul tersusun dan berjalan tanpa galat impor. 2. Seluruh keadaan aplikasi berada di satu objek state. 3. Tampilan digambar dari state, bukan disimpan di DOM. 4. Pilihan terakhir tetap ada setelah halaman dimuat ulang. 5. Simpanan yang rusak tertangani, halaman tetap terbuka.

Aturan pengerjaan

Hal	Ketetapan
Bentuk kerja	Individu. Satu mahasiswa, satu halaman, satu topik.
Perkakas	VS Code + peramban Chrome atau Firefox. Node.js tidak dipakai hari ini — npm dan Vite hanya diperagakan dosen.
Berkas awal	Hasil pekerjaan Anda sendiri dari Pertemuan 10, yaitu profil.html, seluruh berkas CSS, js/app.js, js/dom.js, dan js/api.js, disalin ke folder worksheet-p11/. Tidak ada berkas contoh.
Waktu	90 menit di kelas. Bila belum selesai, dilanjutkan di luar kelas paling lambat pukul 23.59 pada hari yang sama.
Penyerahan	Seluruh isi worksheet-p11/ diunggah ke repositori GitHub Anda sendiri, di dalam folder semester ~/pabw/. Periksa halaman repositori Anda, bukan hanya folder di laptop.

Anggaran waktu 90 menit

Menit	Lembar	Yang Anda kerjakan	Kalau waktu habis, berhenti di sini
0–18	A	Salin hasil Pertemuan 10 ke worksheet-p11/, periksa api.js, gambar susunan modul	Sketsa modul terisi
18–42	B	Pecah menjadi js/data.js, js/state.js, js/events.js; js/app.js jadi perakit	Halaman berjalan sama seperti P10, nol galat impor
42–66	C	Pindahkan tampilan ke render(state), tambah tombol favorit; keadaan keluar dari DOM	Ganti kategori hanya mengubah state
66–92	D	Simpan dan muat dengan localStorage dan JSON, nilai bawaan, tombol atur ulang	Muat ulang halaman, pilihan terakhir tetap ada
92–118	E	Tiga kasus sulit: simpanan rusak, versi skema lama, data yang tidak boleh disimpan	Tiga kasus ditangani + catatan apa yang tidak disimpan
118–150	F	Periksa satu per satu, satu baris untuk diingat, tiket keluar, penilaian mandiri, README, catat PR Node.js	Lembar F terisi dan tersimpan

Selalu kerjakan berurutan. Setiap lembar mengandaikan lembar sebelumnya sudah selesai. Kolom paling kanan bukan batas waktu — hanya penunjuk laju.

Lembar A — Memeriksa berkas dan menggambar susunan modul

BERKAS YANG TERLALU PANJANG ADALAH BERKAS YANG BELUM DIPECAH

Selama enam pertemuan berkas JavaScript Anda tumbuh terus. Hari ini berkas itu dipotong menjadi beberapa berkas, dan setiap berkas mendapat satu tanggung jawab. Lembar ini memeriksa bahan yang Anda bawa, sebelum ada satu baris pun yang dipecah.

A.1 Berkas yang harus ada sebelum Anda mulai

Salin seluruh hasil Pertemuan 10 ke folder worksheet-p11/. Periksa satu per satu, karena pekerjaan hari ini bertumpu pada berkas-berkas itu, dan yang hilang akan terasa sebagai galat impor di lembar B.

Berkas	Dari	Ada?	Jumlah baris
profil.html	P9, P10		
Seluruh berkas CSS (enam berkas)	P4–P6		
js/app.js	P8		
js/dom.js	P9, P10		
js/api.js	P10		

A.2 Periksa js/api.js dari Pertemuan 10

Pertemuan 10 menghasilkan js/api.js dengan fungsi ambilProyek. File itu dipakai apa adanya hari ini. Periksa juga apakah ada file lain di folder js/ yang namanya sama dengan file yang akan Anda buat hari ini. Isi tabel ini apa adanya; kalau tidak ada file lain, tulis "tidak ada".

Nama berkas di js/	Apakah namanya sama dengan yang akan saya buat hari ini?	Yang saya lakukan

Bila ada nama yang bentrok, jangan menimpa. Beri keterangan pada berkas lama, misalnya state-api.js, lalu pakai nama yang baru untuk pekerjaan hari ini. Cara ini membuat kedua pekerjaan tetap bisa dibuka.

A.3 Menghitung berapa banyak baris yang menyentuh halaman

Sebelum memecah, hitung dulu. Buka js/app.js dan js/dom.js, lalu hitung berapa baris yang menyentuh halaman — yaitu baris yang memakai querySelector, createElement, append, classList, atau textContent. Angka itu menunjukkan seberapa besar bagian tampilan yang harus keluar dari berkas data.

Berkas	Jumlah baris seluruhnya	Baris yang menyentuh halaman
js/app.js		
js/dom.js		

A.4 Susunan modul yang akan saya buat

Isi tabel ini sebelum menulis kode. Satu berkas, satu tanggung jawab — kalau satu berkas punya dua pekerjaan, pecahlah.

Berkas	Tanggung jawabnya	Yang dipindah dari mana

Berkas	Tanggung jawabnya	Yang dipindah dari mana
js/data.js	Objek profil dan fungsi murni (buatPerkenalan, formatKeahlian, saring); daftar proyek tetap di data/proyek.json	js/app.js
js/state.js	Objek state tunggal dan fungsi pengubahnya	js/app.js
js/dom.js	Seluruh sentuhan ke halaman: pemilih, render, kelas	js/app.js dan berkas js/dom.js lama
js/events.js	Pemasangan seluruh pendengar peristiwa	js/app.js
js/app.js	Perakit: mengimpor semuanya dan menjalankan aplikasi	

Sudah dikerjakan	Tanda
Folder worksheet-p11/ berisi profil.html, seluruh CSS, js/app.js, dan js/dom.js	✓
js/api.js dari Pertemuan 10 sudah diperiksa, dan nama yang bentrok sudah diberi keterangan	
Saya sudah menghitung berapa baris yang menyentuh halaman	
Tabel A.4 terisi lengkap, dan tidak ada berkas dengan dua tanggung jawab	✓
Halaman dibuka lewat server lokal dan Console masih bersih seperti di P10	✓

Jangan memecah berkas sambil menambah fitur. Selain satu tombol favorit di lembar C, pertemuan ini tidak menambah fitur apa pun. Yang berubah hanya tempat kode itu ditulis. Bila perilaku halaman berubah, ada yang salah dalam pemecahan, bukan fitur baru yang kurang.

SIMPAN KE GIT — SETELAH LEMBAR A

```
git add .
git commit -m "pesan Anda"
git push

# pesan commit yang disarankan:
# P11 — periksa bahan, gambar rencana
```

Lembar A sudah selesai? Simpan hasilnya sekarang, jangan menunggu kelas berakhir. Simpanan ini tidak memerlukan langkah tambahan apa pun — hanya memastikan pekerjaan Anda tidak bergantung pada satu berkas di satu komputer.

A — simpan perubahan

Cara memastikan: buka Console. Halaman berjalan persis seperti akhir Pertemuan 10, tanpa satu galat pun.

Lembar B — Memecah berkas dan menyambungkannya kembali

SATU BERKAS, SATU TANGGUNG JAWAB

Lembar ini adalah inti pertemuan. Lima berkas dibuat, dan setiap berkas hanya boleh tahu pekerjaannya sendiri. app.js berhenti berisi logika; ia hanya merakit berkas lain.

B.1 export dan import, serta jalur yang wajib

Setiap modul punya lingkupnya sendiri: yang tidak Anda ekspor tidak bisa dilihat berkas lain. Jalur relatif pada import wajib diawali ./ dan wajib diakhiri .js — peramban tidak menambahkan akhiran itu sendiri. File pertama yang dibuat adalah js/data.js. Ingat isi app.js di Pertemuan 8: objek profil, array daftarProyek, dan dua fungsi, buatPerkenalan dan formatKeahlian. Sejak Pertemuan 10, daftarProyek sudah pindah ke data/proyek.json dan diambil lewat api.js, jadi tidak ditulis lagi di data.js. Yang masih tertinggal di app.js adalah objek profil dan kedua fungsi itu; ketiganya dipindah ke data.js, ditambah saring() di lembar C. Setelah itu app.js benar-benar hanya merakit.

```
// js/data.js: sisa P8 yang tidak diambil dari server
export const profil = { nama: "...", peran: "...", keahlian: [ /* ... */ ] };
export function buatPerkenalan({ nama, peran }) { /* ... dari P8 */ }
export const formatKeahlian = (daftar) => daftar.join(" · ");
export function saring(daftar, kategori) { /* ... ditulis di lembar C */ }
// daftarProyek TIDAK di sini: ia ada di data/proyek.json sejak P10

// js/dom.js: memakai isi data.js
import { profil, buatPerkenalan, formatKeahlian, saring } from './data.js';
//       nama di dalam { } sama persis dengan yang di-export;
//       jalur wajib diawali ./ dan diakhiri .js
```

Bentuk	Ditulis di berkas asal	Diimpor dengan
Named export	export const state = { ... }	import { state } from './state.js'
Named function	export function formatKeahlian() {}	import { formatKeahlian } from './data.js'
Default export	export default ... (tidak dipakai hari ini)	cukup dikenali; semua file hari ini memakai named export
Yang tidak diekspor	const bantu = ...	Tidak bisa diimpor berkas lain sama sekali

B.2 Satu baris di profil.html yang tidak boleh terlupa

Tanpa type="module", peramban membaca berkas itu sebagai skrip biasa dan berhenti pada kata export pertama. Hanya satu baris pemanggil di profil.html; berkas lain dipanggil lewat import. Sejak Pertemuan 10 profil.html sudah hanya memuat app.js. Pastikan masih begitu: bila baris <script> untuk dom.js dari Pertemuan 9 masih ada, hapus. dom.js, state.js, events.js, data.js, dan api.js semuanya dipanggil lewat import, bukan lewat tag script.

```
<!-- profil.html — hanya satu baris ini yang berubah -->

<script type="module" src="js/app.js"></script>
```

B.3 Lima berkas, lima tanggung jawab

Kerjakan satu berkas sampai selesai, lalu periksa Console sebelum menambah berkas berikutnya. Cara itu membuat galat impor mudah dilacak — saat baru satu berkas dipindahkan, satu-satunya sebab galat pasti berkas itu.

Langkah	Yang dilakukan	Console harus

Langkah	Yang dilakukan	Console harus
1. data.js	Pindahkan objek profil, buatPerkenalan, dan formatKeahlian dari app.js; daftar proyek tetap di data/proyek.json	Bersih; app.js belum diubah
2. state.js	Buat objek state dan fungsi pengubahnya; daftar proyek mula-mula [], lalu diisi isiProyek()	Bersih
3. events.js	Pindahkan seluruh addEventListener dari app.js dan dom.js	Bersih; halaman belum berubah perilaku
4. dom.js	Sisakan hanya pemilih, render, dan perubahan kelas	Bersih; halaman tampil sama seperti P10
5. app.js	Hapus sisa logika; sisakan import dan pemanggilan	Bersih; halaman berperilaku persis seperti P10

B.4 Menyambungkan js/api.js dari Pertemuan 10

Impor file itu apa adanya, jangan diubah, dan jangan dihitung sebagai salah satu dari lima modul. Data dari server masuk ke state lewat fungsi pengubah di state.js, bukan ditulis langsung dari app.js. Blok try/catch serta pesan memuat dan gagal dari Pertemuan 10 tetap dipertahankan di app.js. Baris pulihkan(muat()) baru bekerja setelah lembar D selesai; sampai saat itu boleh dijadikan komentar dulu.

```
// state.js: data dari server masuk lewat fungsi pengubah
export function isiProyek(daftar) {
  state.daftarProyek = daftar;
  render(state);
}

// app.js: perakit. Di P10 fungsi ini bernama muat(); di P11
// namanya mulai(), karena nama muat() dipakai untuk localStorage (lembar D).
import { ambilProyek } from './api.js';
import { isiProyek, pulihkan, muat } from './state.js';
import { tampilkanGalat, aturMemuat } from './dom.js';
import { pasangPendengar } from './events.js';

async function mulai() {
  pulihkan(muat());              // lembar D: pilihan terakhir, atau nilai bawaan
  pasangPendengar();               // events.js: klik filter, favorit, atur ulang
  aturMemuat(true);
  try {
    isiProyek(await ambilProyek()); // isiProyek memanggil render(state)
  } catch (galat) {
    tampilkanGalat('Data gagal dimuat. Periksa koneksi, lalu muat ulang.');
  } finally {
    aturMemuat(false);
  }
}
mulai();
```

B.5 Periksa hasil pemecahan

Yang diperiksa	Hasil yang benar	Hasil yang saya dapat
Console	Bersih, tidak ada pesan merah maupun kuning	Bersih, tidak ada pesan error.
Perilaku halaman	Sama persis seperti akhir Pertemuan 10	Sama
Isi app.js	Hanya import dan pemanggilan; tidak ada logika	Sesuai
Jumlah modul	Lima berkas wajib; api.js dari P10 dihitung terpisah	Ada 5 berkas

Yang diperiksa	Hasil yang benar	Hasil yang saya dapat
Jalur import	Semua berawalan ./ dan berakhiran .js	sesuai

Yang dinilai adalah pemecahan lima modul wajib, bukan jumlah berkas. Dengan api.js dari Pertemuan 10, totalnya enam file, dan itu benar. Yang tidak boleh adalah satu file memikul dua tanggung jawab.

SIMPAN KE GIT — SETELAH LEMBAR B

```
git add .
git commit -m "pesan Anda"
git push

# pesan commit yang disarankan:
# P11 — lima modul berjalan
```

Lembar B sudah selesai? Simpan hasilnya sekarang, jangan menunggu kelas berakhir. Simpanan ini tidak memerlukan langkah tambahan apa pun — hanya memastikan pekerjaan Anda tidak bergantung pada satu berkas di satu komputer.

B — simpan perubahan

Cara memastikan: muat ulang halaman. Tampilannya sama seperti kemarin, Console bersih, dan tidak ada satu pun pesan Failed to resolve module specifier.

Lembar C — Memindahkan keadaan keluar dari halaman

TAMPILAN ADALAH FUNGSI DARI KEADAAN

Sampai lembar B, keadaan aplikasi Anda masih menempel di halaman: kategori aktif tersimpan sebagai kelas pada tombol. Selama halaman tidak pernah digambar ulang, cara itu bekerja — dan itulah jebaknya.

C.1 Mengapa keadaan yang menempel di halaman akhirnya kacau

Begitu daftar digambar ulang, seluruh elemen lama dibuang dan yang baru belum membawa keadaan apa pun. Nilai yang tersimpan di halaman ikut hilang, dan halaman tampak "lupa".

Keadaan menempel di DOM	Keadaan di satu objek
Dibaca dari kelas aktif tombol: document.querySelector(".aktif")	Dibaca dari state.kategoriAktif
Hilang setiap kali wadah digambar ulang	Bertahan, karena tidak berada di halaman
Bisa berbeda dari tampilan yang terlihat	Satu-satunya sumber: tidak ada salinan kedua
Sulit diuji karena butuh halaman	Bisa diperiksa cukup dengan melihat isi satu objek state

C.2 render(data) menjadi render(state)

Ubah fungsi render Anda supaya menerima objek state, bukan array mentah. Setelah itu render punya tiga pekerjaan tetap, dan urutannya tidak boleh ditukar: kosongkan wadahnya, periksa keadaan kosong, lalu isi ulang dari state. Fungsi saring() ditulis di data.js karena ia fungsi murni: menerima daftar dan kategori, lalu mengembalikan daftar yang lolos filter. Dengan begitu dom.js tidak perlu mengimpor dari state.js, dan tidak ada dua file yang saling mengimpor. Fungsi pasangFilter() dari Pertemuan 10 tidak dipakai lagi: pendengar kliknya pindah ke events.js dan hanya memanggil pilihKategori(), sedangkan penyaringannya pindah ke saring() di data.js.

```
function render(state) {
  wadah.textContent = "";                          // 1. kosongkan
  const tampil = saring(state.daftarProyek, state.kategoriAktif);  // dari data.js
  if (tampil.length === 0) {                       // 2. keadaan kosong
    kosong.hidden = false;
    return;
  }
  kosong.hidden = true;
  tampil.forEach((item) =>                         // 3. isi ulang
    wadah.append(buatKartu(item, state.favorit)));
}
```

C.3 Satu tombol kecil: tanda favorit

Hari ini hanya ada satu tambahan di tampilan: tombol ☆ di setiap kartu, supaya pengguna bisa menandai proyek yang disukai. Tombol ini diperlukan karena state Anda perlu menyimpan sebuah array (daftar favorit), bukan hanya satu teks (kategori aktif). Klik pada tombol ditangani dengan satu pendengar di wadah daftar, sama seperti tombol filter di Pertemuan 9.

```
// dom.js: buatKartu() dari P9, ditambah satu tombol
export function buatKartu(proyek, favorit) {
  const li = document.createElement("li");
  li.className = "kartu";
  li.textContent = proyek.judul;
  const tombol = document.createElement("button");
  tombol.className = "favorit";
  tombol.dataset.id = proyek.id;
  tombol.textContent = favorit.includes(proyek.id) ? "★" : "☆";
  li.append(tombol);
  return li;
}

// events.js: di dalam export function pasangPendengar()
document.querySelector("#daftar").addEventListener("click", (event) => {
  const tombol = event.target.closest(".favorit");
  if (!tombol) return;
  tambahFavorit(Number(tombol.dataset.id));
});

// state.js: salinan baru, bukan push
export function tambahFavorit(id) {
  if (state.favorit.includes(id)) return;
  state.favorit = [...state.favorit, id];
  render(state);
}
```

Setiap proyek perlu punya id yang unik. Sejak Pertemuan 10 setiap butir di data/proyek.json sudah punya id; periksa sekali lagi. Nilai dataset selalu berupa teks, karena itu id diubah dulu dengan Number() bila id di data Anda berupa angka.

C.4 Fungsi pengubah: satu pintu masuk untuk setiap perubahan

Tidak ada bagian lain yang boleh mengubah state langsung. Setiap perubahan ditulis sebagai fungsi bernama di state.js, dan fungsi itu memanggil render. Dengan begitu pertanyaan "mengapa tampilan berubah?" selalu punya satu jawaban. Penanda tombol aktif dari Pertemuan 9 (tandaiTombolAktif) tetap dipakai, tetapi pindah ke dom.js dan dipanggil dari render(state). Syaratnya diubah dari tombol === tombolAktif menjadi tombol.dataset.kategori === state.kategoriAktif. Tanpa perubahan itu, setelah halaman dimuat ulang pilihannya pulih dari simpanan, tetapi tombolnya tidak terlihat aktif.

Jangan ditulis di events.js	Tulis di state.js	Alasannya
state.kategoriAktif = "web"	export function pilihKategori(nilai) { … }	Satu tempat yang perlu diperiksa ketika tampilan tidak berubah
state.favorit.push(id)	export function tambahFavorit(id) { … }	Salinan baru dibuat di dalam, data asli tidak diubah di tempat
render() dipanggil dari banyak tempat	render dipanggil di dalam fungsi pengubah	Tidak ada keadaan berubah tanpa tampilan ikut berubah

C.5 Periksa hasil kerja Anda

Yang diperiksa	Hasil yang benar	Hasil yang saya dapat
Ganti kategori	Hanya state.kategoriAktif yang berubah	Sesuai
state (ketik di Console, lihat catatan di bawah)	Seluruh keadaan terlihat dalam satu objek	Sesuai
Cari di berkas	Tidak ada pembacaan keadaan dari DOM	Sesuai
Gambar ulang	Pilihan tetap benar setelah render dipanggil ulang	Sesuai

Tanda keadaan sudah benar-benar keluar dari halaman: klik tombol kategori yang sama dua kali berturut-turut. Tampilannya tidak berubah maupun berlipat.

SIMPAN KE GIT — SETELAH LEMBAR C

```
git add .
git commit -m "pesan Anda"
git push

# pesan commit yang disarankan:
# P11 — keadaan keluar dari halaman
```

Lembar C sudah selesai? Simpan hasilnya sekarang, jangan menunggu kelas berakhir. Simpanan ini tidak memerlukan langkah tambahan apa pun — hanya memastikan pekerjaan Anda tidak bergantung pada satu berkas di satu komputer.

C — simpan perubahan

Cara memastikan: tambahkan sementara baris window.state = state; di akhir app.js, muat ulang halaman, lalu ketik state di Console. Fungsi dan variabel di dalam file module tidak bisa dipanggil langsung dari Console, karena itu perlu baris sementara ini. Seluruh keadaan yang bisa berubah terlihat di sana, dan tidak satu pun disimpan pada elemen. Hapus baris window.state sebelum diserahkan.

Lembar D — Halaman yang mengingat pilihan

HALAMAN YANG MENGINGAT LEBIH NYAMAN DIPAKAI

Sampai lembar C, keadaan Anda benar tetapi tidak bertahan: begitu halaman dimuat ulang, semuanya kembali ke nilai awal. Lembar ini membuat pilihan terakhir bertahan — dan hanya yang memang perlu diingat.

D.1 localStorage hanya menerima teks

localStorage menyimpan teks, bukan objek. Karena itu objek atau array harus diubah menjadi teks lebih dulu dengan JSON.stringify, lalu diubah kembali dengan JSON.parse saat dibaca. Melewatkan langkah itu membuat isinya tersimpan sebagai [object Object]. Tulis KUNCI, VERSI, dan ketiga fungsi di bawah ini di state.js. VERSI cukup ditulis sekali di sini dan ikut disimpan, karena di lembar E nomor itulah yang diperiksa saat memuat.

```
// js/state.js
const KUNCI = "pabw-p11";
const VERSI = 2;

export function simpan(state) {
  const isi = { versi: VERSI,
                kategoriAktif: state.kategoriAktif,
                favorit: state.favorit };
  localStorage.setItem(KUNCI, JSON.stringify(isi));
}

export function muat() {
  try { return JSON.parse(localStorage.getItem(KUNCI)); }
  catch { return null; }        // simpanan rusak: pakai nilai bawaan
}

export function pulihkan(isi) {   // dipanggil sekali dari mulai() di app.js
  if (!isi) return;               // kunjungan pertama: nilai bawaan tetap
  state.kategoriAktif = isi.kategoriAktif;
  state.favorit = isi.favorit;
}

export function aturUlang() {
  localStorage.removeItem(KUNCI);
  state.kategoriAktif = 'semua';
  state.favorit = [];
  render(state);
}
```

Yang dipakai	Perannya	Kapan dijalankan
localStorage.setItem(...)	Menyimpan teks dengan satu nama kunci	Sesudah keadaan berubah dan sebelum render
localStorage.getItem(...)	Membaca kembali; null bila belum pernah disimpan	Sekali saja, saat halaman pertama dimuat
JSON.stringify(objek)	Mengubah objek menjadi teks	Seluruh isi yang berupa objek atau array
JSON.parse(teks)	Mengubah teks kembali menjadi objek	Segera sesudah membaca, dibungkus try/catch
localStorage.removeItem(...)	Menghapus simpanan	Tombol atur ulang

D.2 Nilai bawaan pada kunjungan pertama

Pada kunjungan pertama getItem mengembalikan null, bukan galat. Nilai itu harus diganti dengan nilai bawaan sebelum dipakai — dan pemeriksaan null itu wajib ada, bukan pilihan. Setelah itu, tambahkan simpan(state) di setiap fungsi pengubah, tepat sebelum render(state), supaya setiap perubahan langsung tersimpan. Tombol atur

ulang juga dibuat sekarang, karena dinilai di rubrik: tambahkan <button id="atur-ulang">Atur ulang</button> di profil.html, lalu di events.js pasang satu pendengar klik yang memanggil aturUlang().

Keadaan	Nilai bawaan yang saya pakai	Alasannya
Kategori aktif	'semua'	Agar saat pengguna pertama kali membuka halaman, semua daftar lagu dari berbagai kategori langsung ditampilkan (tidak ada yang disembunyikan).
Daftar favorit	[] *(Array kosong)*	Karena pengguna baru belum memilih atau menandai lagu favorit apa pun, sehingga daftar simpanan bintangnya harus kosong.

D.3 Batas penyimpanan klien

Penyimpanan klien mudah dipakai dan mudah disalahgunakan. Batasnya bukan pendapat, melainkan sifat peramban — dan ada hal yang tidak boleh disimpan sama sekali.

Batas	Artinya	Akibatnya bagi pekerjaan Anda
Hanya teks	Angka, objek, dan array harus diubah lebih dulu	Selalu lewat JSON sebelum disimpan
Sekitar 5 MB per alamat	Batasnya per alamat, bukan per berkas	Simpan pilihan, bukan gambar atau data besar
Sinkron dan menghambat	Membaca dan menulis menahan halaman sejenak	Jangan menyimpan pada setiap ketikan
Tidak terenkripsi	Isinya bisa dibaca siapa pun yang memakai komputer itu	Jangan simpan token, kata sandi, atau data pribadi
Terikat alamat	Simpanan satu alamat tidak terlihat oleh alamat lain	Simpanan hilang bila nomor port server lokal berubah

D.4 Yang saya simpan, dan yang sengaja tidak

Keadaan	Disimpan?	Alasannya
Kategori aktif	Ya	Agar pilihan filter terakhir dari pengguna (misalnya sedang melihat kategori "Pop") tetap diingat saat halaman dimuat ulang.
Daftar favorit	Ya	Agar lagu-lagu yang sudah ditandai bintang (disukai) tidak hilang meskipun peramban atau tab ditutup. Ini adalah fungsi utama memori klien.
Data proyek dari server	tidak	Data server bisa diperbarui sewaktu-waktu oleh pemilik web. Jika disimpan di memori peramban, datanya bisa menjadi basi (*outdated*) dan hanya akan membuang-buang kapasitas *LocalStorage*.
Token atau data pribadi	Tidak	Tidak terenkripsi dan bisa dibaca siapa pun

Uji dengan cara yang paling meyakinkan: ubah pilihannya, tutup tab, lalu buka lagi. Bukan sekadar muat ulang — tutup tab benar-benar, karena simpanan harus bertahan melewati penutupan tab.

SIMPAN KE GIT — SETELAH LEMBAR D

```
git add .
git commit -m "pesan Anda"
git push

# pesan commit yang disarankan:
# P11 — halaman mengingat
```

Lembar D sudah selesai? Simpan hasilnya sekarang, jangan menunggu kelas berakhir. Simpanan ini tidak memerlukan langkah tambahan apa pun — hanya memastikan pekerjaan Anda tidak bergantung pada satu berkas di satu komputer.

D — simpan perubahan

Cara memastikan: ubah kategori dan tambahkan satu favorit, tutup tabnya, lalu buka lagi. Pilihan terakhir Anda masih ada.

Lembar E — Simpanan yang rusak dan jalan pulangnya

SIMPANAN BISA RUSAK, SIAPKAN JALAN PULANGNYA

Setelah data mulai disimpan, kegagalan berpindah tempat: bukan lagi di kode Anda, melainkan pada isi simpanan yang sudah lebih dulu berbeda dari dugaan. Tiga bentuk kegagalan di bawah ini paling sering terjadi, dan ketiganya bisa Anda buat sekarang.

E.1 Tiga cara simpanan menjadi rusak

Yang dinilai bukan keberhasilan mencegah kerusakan — kerusakan itu datang dari luar — melainkan apakah halaman Anda tetap terbuka ketika kerusakan itu terjadi.

Kasus	Artinya	Yang dilakukan
Simpanan rusak atau null	Isinya bukan JSON yang sah, sehingga JSON.parse melempar galat dan skrip berhenti	Bungkus pembacaan dengan try/catch; bila gagal, buang simpanan itu dan pakai nilai bawaan
Versi skema lama	Bentuk isi berubah antarpertemuan, sehingga properti yang dicari tidak ada	Sertakan nomor versi pada isi simpanan; bila versinya berbeda, buang dan mulai dari nilai bawaan
Data yang tergoda disimpan	Token atau data pribadi ikut tersimpan karena mudah	Simpan hanya pilihan tampilan, dan sebutkan dalam catatan apa yang sengaja tidak disimpan

E.2 Nomor versi pada isi simpanan

Nomor versi adalah satu properti tambahan yang menyebut bentuk isi simpanan. Tanpa itu, isi bentuk lama akan tetap dipakai dan propertinya hilang satu per satu tanpa pesan galat. Periksa pada saat memuat, bukan pada saat menyimpan. Yang berubah di bawah ini bukan fungsi tambahan: fungsi muat() dari D.1 diganti dengan versi ini, jangan ditulis sebagai fungsi kedua. VERSI tidak ditulis ulang, karena sudah ada di D.1.

```
// js/state.js: muat() dari D.1 diganti dengan ini
export function muat() {
  try {
    const isi = JSON.parse(localStorage.getItem(KUNCI));
    if (isi?.versi !== VERSI) {
      localStorage.removeItem(KUNCI);   // bentuk lama: buang
      return null;
    }
    return isi;
  } catch {
    localStorage.removeItem(KUNCI);     // isi rusak: buang
    return null;
  }
}
```

E.3 Uji kerusakannya sendiri

Jangan menunggu kerusakan terjadi. Buat kerusakannya di Console, muat ulang halaman, lalu pastikan halaman Anda tetap terbuka. Ini pemeriksaan yang paling cepat dan paling sering dilewatkan.

Yang ditulis di Console	Yang harus terjadi	Hasil yang saya dapat
localStorage.setItem("pabw-p11", "bukan-json")	Halaman tetap terbuka; simpanan dibuang; nilai bawaan dipakai	
localStorage.setItem("pabw-p11", "{}")	Versi tidak cocok; simpanan dibuang; nilai bawaan dipakai	
localStorage.getItem("pabw-p11") (setelah muat ulang)	null karena simpanan rusak sudah dibuang; tidak ada galat yang menghentikan skrip	

E.4 Catatan: apa yang sengaja tidak saya simpan

Yang tidak disimpan	Alasan saya
Data token, kata sandi, atau data pribadi	Token dan kata sandi tidak terenkripsi di peramban
Data daftar proyek/lagu dari server (api.js)	Data dari server bisa diperbarui kapan saja. Menyimpannya di memori klien (*LocalStorage*) akan membuat data menjadi usang (*outdated*) jika ada pembaruan di server, serta membuang kapasitas memori peramban.

Simpan dua tangkapan layar: satu saat simpanan rusak sengaja dibuat, satu setelah halaman pulih dengan nilai bawaan. Ketiganya Anda perlukan di lembar F.

SIMPAN KE GIT — SETELAH LEMBAR E

```
git add .
git commit -m "pesan Anda"
git push

# pesan commit yang disarankan:
# P11 — jalan pulang tersedia
```

Lembar E sudah selesai? Simpan hasilnya sekarang, jangan menunggu kelas berakhir. Simpanan ini tidak memerlukan langkah tambahan apa pun — hanya memastikan pekerjaan Anda tidak bergantung pada satu berkas di satu komputer.

E — simpan perubahan

Cara memastikan: rusakkan simpanannya sendiri dari Console. Halaman tetap terbuka, dan Console tidak menampilkan galat yang belum tertangani.

TIKET KELUAR

Dikerjakan sendiri, tanpa berdiskusi. Lembar ini mengukur pemahaman Anda, bukan kecepatan Anda menyalin jawaban teman.

1. Sebutkan satu berkas yang Anda buat hari ini, tanggung jawabnya, dan dua berkas lain yang mengimpornya.

Berkas js/state.js. Tanggung jawabnya adalah menyimpan objek keadaan (*state*) tunggal dan mengelola semua fungsi yang mengubah keadaan tersebut. Berkas ini diimpor oleh js/events.js (untuk memanggil fungsi pengubah saat ada klik) dan js/app.js (untuk memuat keadaan saat aplikasi pertama kali berjalan).

2. Mengapa jalur import wajib berawalan ./ dan berakhiran .js? Apa yang muncul di Console bila keduanya dilewatkan?

Awalan ./ diperlukan agar peramban mengenali bahwa itu adalah berkas lokal (relatif), dan akhiran .js wajib karena *ES Modules* di peramban membutuhkan nama berkas yang pasti dan lengkap beserta ekstensinya. Jika dilewatkan, Console akan menampilkan pesan galat (*error*) tipe TypeError: Failed to resolve module specifier atau galat 404 (Not Found) karena peramban gagal menemukan lokasi berkasnya.

3. Sebelum hari ini, di mana keadaan kategori aktif disimpan? Mengapa cara itu akhirnya kacau saat daftar digambar ulang?

Sebelumnya, keadaan disimpan pada tampilan (DOM), misalnya dengan mengecek tombol mana yang memiliki kelas CSS .aktif. Cara ini menjadi kacau karena ketika daftar digambar ulang (dikosongkan dan diisi ulang), status pilihan tersebut bisa terhapus, sehingga logika aplikasi dan antarmuka tercampur aduk dan sulit dikendalikan.

4. Apa yang harus dilakukan sebelum dipakai, ketika getItem mengembalikan null pada kunjungan pertama?

Saat getItem mengembalikan null, kita harus menggantinya dengan nilai bawaan (*default values*) terlebih dahulu sebelum aplikasi dirender. Misalnya, mengatur kategori aktif menjadi 'semua' dan daftar favorit menjadi *array* kosong []

5. Mengapa simpanan yang rusak harus tetap membuat halaman terbuka? Tulis cara Anda menanganinya.

Kerusakan memori pilihan pengguna (seperti kategori terakhir) tidak boleh menghalangi fitur utama aplikasi, yaitu menampilkan daftar proyek/lagu. Penanganannya dilakukan menggunakan blok **try...catch** saat memuat data. Jika terjadi galat (masuk ke catch), kita menghapus simpanan yang rusak tersebut dengan localStorage.removeItem() lalu mengembalikan aplikasi ke nilai bawaan agar halaman tetap berfungsi normal.

6. Sebutkan satu hal yang bisa Anda simpan di localStorage dan satu hal yang tidak boleh. Jelaskan bedanya.

Bisa disimpan: Pilihan kategori aktif atau daftar favorit.

Tidak boleh disimpan: Kata sandi, token autentikasi, atau data rahasia.

Bedanya: Pilihan pengguna aman disimpan karena hanya berfungsi untuk kenyamanan tampilan dan tidak membahayakan sistem. Sebaliknya, kata sandi dan token dilarang disimpan di localStorage karena tempat ini sama sekali tidak terenkripsi, sehingga datanya bisa disadap atau dicuri oleh peretas.

F.1 Periksa satu per satu

Jangan menyerahkan pekerjaan sebelum menjalankan daftar ini. Semua yang diperiksa di bawah bisa Anda lihat sendiri dalam lima menit, dan semuanya benar-benar dinilai.

- ✓☐ Berkas lima modul ada di repositori: data.js, state.js, dom.js, events.js, app.js — ditambah js/api.js dari P10
- ✓☐ Impor seluruh jalur berawalan ./ dan berakhiran .js; Console bersih tanpa satu pun galat impor
- ✓☐ Perilaku halaman bekerja persis seperti akhir Pertemuan 10, ditambah tombol favorit; tidak ada fitur lain yang ditambahkan
- ✓☐ State seluruh keadaan yang bisa berubah berada di satu objek state, dan tidak ada keadaan yang dibaca kembali dari DOM
- ✓☐ Pengubah setiap perubahan keadaan ditulis sebagai fungsi bernama di state.js, dan fungsi itulah yang memanggil render
- ✓☐ Simpanan pilihan bertahan setelah tab ditutup; JSON dipakai; nilai bawaan terpasang; tombol atur ulang bekerja
- ✓☐ Simpanan rusak try/catch dipakai, nomor versi diperiksa, dan saya sudah mencoba merusaknya sendiri dari Console
- ✓☐ Deklarasi AI bagian README menyebutkan bagian mana yang dibantu AI dan bagian mana yang saya kerjakan sendiri
- ✓☐ Git ada lebih dari satu commit hari ini, dan pesan commit-nya menjelaskan isinya

F.2 Pekerjaan rumah sebelum Pertemuan 12

Pasang Node.js LTS di laptop Anda, bukan versi Current. Buktikan node -v dan npm -v berjalan di terminal, lalu catat dua nomor versinya di README bagian Pertemuan 11. Bila perintahnya tidak dikenali: tutup dan buka ulang terminal, periksa PATH, pastikan yang terpasang versi LTS.

Perintah	Versi yang muncul	Catatan bila gagal
node -v		
npm -v		

F.3 Satu baris untuk diingat

Tulis satu kalimat — bukan untuk dosen, untuk Anda sendiri dua pekan dari sekarang — tentang hal yang hari ini paling sering membuat Anda salah.

F.4 Penilaian mandiri

Isi perkiraan nilai Anda apa adanya sebelum menyerahkan lembar ini. Kolom Bukti diisi nama berkas atau baris kode yang bisa dibuka dosen.

☐ Pengungkapan AI README menyebutkan bagian mana yang dibantu AI dan bagian mana yang saya kerjakan sendiri

☐ Keaslian topik dan isi halaman ini milik saya sendiri, tidak menyalin pekerjaan mahasiswa lain

Bagian	Bobot	Nilai saya	Bukti
Pemecahan modul dan impor	25	25	
Pemisahan state dari tampilan	25	15	
Simpan dan muat localStorage	25	25	
Penanganan simpanan rusak	15	15	
Kebersihan kode, deklarasi AI, dan bukti	10	10	
TOTAL	100	90	

F.5 Catatan untuk pengampu

Bagian tersulit hari ini:

Yang ingin saya bahas di kelas berikutnya:

Berkas tambahan yang saya bawa dari Pertemuan 10:

Tanda tangan: Muchamad adi prsetyo Tanggal: 19/10/2026 Jam selesai: 17:09

SIMPAN KE GIT — SETELAH LEMBAR F

```
git add .
git commit -m "pesan Anda"
git push

# pesan commit yang disarankan:
# P11 — periksa, lalu simpan
```

Lembar F sudah selesai? Simpan hasilnya sekarang, jangan menunggu kelas berakhir. Simpanan ini tidak memerlukan langkah tambahan apa pun — hanya memastikan pekerjaan Anda tidak bergantung pada satu berkas di satu komputer.

F — simpan perubahan

Cara memastikan: buka README di repositori Anda. Bagian Pertemuan 11 sudah memuat deklarasi AI dan dua nomor versi Node.js, dan commit terakhir adalah commit terbaru.

Lampiran A — Rubrik penilaian

RUBRIK PENILAIAN total 100

Aspek	Bobot	Nilai 100 — sangat baik	Nilai 65 — memadai	Nilai 35 — kurang
Pemecahan modul dan impor	25%	Lima modul tersusun dengan satu tanggung jawab masing-masing; app.js hanya merakit; seluruh jalur impor benar; Console bersih	Modul terpisah, tetapi satu berkas masih memikul dua tanggung jawab, atau perilaku halaman berubah dari P10	Berkas belum dipecah, atau halaman tidak berjalan karena galat impor
Pemisahan state dari tampilan	25%	Seluruh keadaan berada di satu objek state; perubahan hanya lewat fungsi pengubah; tidak ada keadaan yang dibaca kembali dari DOM	State sudah ada, tetapi sebagian perubahan masih ditulis langsung dari events.js atau nilai keadaan masih disimpan pada elemen	Keadaan masih menempel di DOM, atau halaman lupa setelah render ulang
Simpan dan muat localStorage	25%	JSON.stringify dan JSON.parse dipakai tepat; nilai bawaan terpasang; pilihan bertahan setelah tab ditutup; tombol atur ulang bekerja	Simpanan bekerja, tetapi nilai bawaan belum dipasang atau tombol atur ulang belum membersihkan simpanannya	Simpanan tidak bertahan, atau objek tersimpan tanpa JSON sehingga isinya [object Object]
Penanganan simpanan rusak	15%	Pembacaan dibungkus try/catch; nomor versi diperiksa; simpanan lama dibuang; halaman tetap terbuka; ada catatan apa yang sengaja tidak disimpan	try/catch ada, tetapi nomor versi belum dipakai atau catatan pengecualian belum ditulis	Simpanan rusak membuat halaman gagal terbuka, atau data pribadi ikut disimpan
Kebersihan kode, deklarasi AI, dan bukti	10%	Penamaan berkas dan fungsi konsisten; tidak ada kode mati dari P9 dan P10; deklarasi AI terisi; lebih dari satu commit dengan pesan yang menjelaskan	Kode rapi tetapi deklarasi AI belum lengkap, atau hanya satu commit	Deklarasi AI kosong, atau pekerjaan tidak tersimpan di repositori

Yang dinilai adalah pekerjaan Anda sendiri. Topik yang berbeda antar mahasiswa membuat penilaian tidak bisa dibandingkan antar pekerjaan — karena itu yang diperiksa adalah apakah setiap bagian bekerja sebagaimana dimaksud, bukan apakah ia sama dengan contoh.

Lampiran B — Rujukan cepat dan kalau macet

REFERENSI CEPAT MODUL, STATE, DAN SIMPANAN hafalkan yang ini

Yang dipakai	Perannya	Kapan dipakai
export / import	Menyerahkan isi modul kepada berkas lain	Saat memecah kode besar menjadi beberapa berkas kecil (modul) agar fungsi dari satu berkas bisa digunakan di berkas lainnya.
jalur './berkas.js'	Menunjuk modul lain secara relatif	Saat memanggil (import) modul lokal yang berada di dalam direktori proyek yang sama, agar peramban bisa menemukannya.

Yang dipakai	Perannya	Kapan dipakai
const state = { }	Menampung seluruh keadaan aplikasi	Di awal program untuk membuat satu wadah utama yang menyimpan semua data sementara (seperti kategori yang dipilih dan daftar favorit).
fungsi pengubah	Satu pintu masuk setiap perubahan keadaan	Saat pengguna melakukan aksi (seperti klik tombol kategori atau bintang) yang membutuhkan pembaruan data di dalam state.
render(state)	Menggambar ulang halaman dari keadaan	Setiap kali fungsi pengubah selesai memperbarui data di state, agar tampilan layar sinkron dengan data terbaru
JSON.stringify	Mengubah objek menjadi teks	Sesaat sebelum menyimpan data state (yang berupa objek/array) ke dalam localStorage.setItem.
JSON.parse	Mengubah teks kembali menjadi objek	Sesaat setelah mengambil data dari localStorage.getItem, agar teks tersebut bisa diolah kembali sebagai objek JavaScript.
localStorage.setItem / getItem	Menyimpan dan membaca teks	setItem dipakai tiap kali state berubah agar tidak hilang. getItem dipakai saat halaman pertama kali dimuat untuk memulihkan ingatan.
localStorage.removeItem	Menghapus simpanan	Saat tombol "Atur Ulang" diklik, atau saat sistem mendeteksi versi data yang tersimpan sudah usang atau rusak (di dalam blok catch).
nilai bawaan ??	Mengganti null pada kunjungan pertama	Saat pengguna baru pertama kali membuka web dan memori perambannya masih kosong (getItem menghasilkan null), agar aplikasi tidak *error*.

KALAU MACET cari gejalanya di sini

Gejala	Penyebab yang paling sering	Yang dilakukan
Failed to resolve module specifier	Jalur import tidak diawali ./ atau tidak berakhiran .js	Tulis './data.js' — peramban tidak menambahkan akhiran sendiri
Console: Unexpected token 'export'	Skrip tidak memakai type="module"	Tambahkan type="module" pada baris <script> pemanggilnya
Console: Cannot use import statement outside a module	Berkas dimuat sebagai skrip biasa, bukan modul	Periksa atribut type pada seluruh baris <script>
Halaman diam setelah dipecah, Console bersih	app.js belum memanggil fungsi dari modul yang baru dibuat	Impor fungsinya, lalu panggil di app.js
Nilai berubah acak antarberkas	Modul mengimpor memutar: A mengimpor B, B mengimpor A	Pindahkan bagian yang dipakai bersama ke modul ketiga
Simpanan tersimpan sebagai [object Object]	Objek disimpan tanpa JSON.stringify	Ubah dulu menjadi teks sebelum disimpan
Halaman gagal terbuka setelah simpanan lama ada	JSON.parse melempar galat dan tidak ditangkap	Bungkus dengan try/catch, buang simpanannya, pakai nilai bawaan
Simpanan tetap ada padahal sudah diatur ulang	removeItem tidak dipanggil, hanya tampilannya yang dikosongkan	Panggil localStorage.removeItem(KUNCI), lalu gambar ulang
Pilihan hilang saat port server berubah	Simpanan terikat pada alamat, termasuk nomor portnya	Pakai satu nomor port tetap selama mengerjakan proyek ini

Gejala	Penyebab yang paling sering	Yang dilakukan
Kartu berlipat setiap kali render	Wadah tidak dikosongkan di baris pertama render	Tambahkan wadah.textContent = "" di awal fungsi render
Versi skema berbeda tanpa pesan galat	Isi bentuk lama dipakai karena tidak ada nomor versi	Sertakan properti versi dan periksa saat memuat

Halaman yang Anda bangun hari ini adalah bahan Pertemuan 12: proyek lima modul ini akan diporting menjadi komponen Vue, lalu dibangun untuk produksi. Simpan pekerjaan ini rapi di repositori Anda, dan jangan lupa memasang Node.js LTS sebelum pertemuan berikutnya.

LAMPIRAN — MENYIMPAN PEKERJAAN ANDA KE GIT sekali saja, bukan tiap pertemuan

Lampiran ini bukan pengantar Git. Panduan Git dan GitHub yang sudah Anda pakai sejak Pertemuan 3 masih berlaku seluruhnya; bagian ini hanya mengulang tiga perintah yang paling sering dipakai, karena mulai hari ini jumlah berkasnya bertambah.

Tiga perintah yang akan Anda pakai terus

Perintah	Artinya	Kapan dijalankan
git add .	Menandai seluruh perubahan di folder ini untuk disimpan	Setiap kali selesai satu lembar
git commit -m "pesan"	Menyimpan perubahan itu sebagai satu riwayat	Langsung setelah git add
git push	Mengirim riwayat itu ke GitHub	Setelah commit, supaya dosen bisa melihatnya

Titik pada git add . Titik itu berarti "seluruh berkas di folder ini". Tanpa titik, Git tidak tahu apa yang harus disimpan. Berkas baru seperti state.js dan events.js ikut tersimpan karena berada di dalam folder semester.

Berapa kali harus menyimpan?

Tidak ada angka yang benar, tetapi patokannya begini: bila Anda sudah menyelesaikan satu hal yang berarti dan bisa Anda jelaskan, simpanlah. Pada pertemuan ini minimal tiga kali: setelah modul terpecah, setelah keadaan keluar dari DOM, dan setelah simpanan bekerja.

Kalau ini terjadi	Yang Anda lakukan
Sudah menulis banyak, belum pernah commit	Simpan sekarang juga
Salah commit, belum push	Perbaiki lalu commit lagi — jangan hapus repositori
Sudah push, lalu menemukan kekeliruan	Perbaiki berkasnya dan commit lagi; riwayat lama tidak hilang dan itu wajar

Yang dinilai bukan jumlah commit. Menyimpan sering membuat pekerjaan Anda tidak bergantung pada satu berkas di satu komputer — dan pada pertemuan ujian, repositori itulah yang menentukan apakah pekerjaan Anda bisa dinilai.